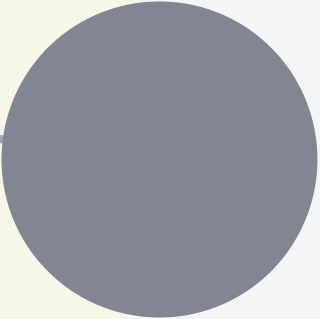




Secure Authentication System

Inkar Khairatkyzy/ MAT364

A lightweight secure authentication system using FastAPI, JWT, TOTP, and AES/RSA cryptography



Problem Statement

Why do we need secure authentication?

- Password leaks are extremely common (weak passwords, reused credentials).
- Phishing attacks trick users into revealing passwords.
- Session hijacking lets attackers steal active login sessions.

Motivation

Understand cryptography fundamentals by building everything manually.
Implement real security components: JWT, RSA, AES, Argon2id, TOTP.



Cryptographic Components Used

Argon2id — password hashing

RSA-2048 — signing JWT access/refresh tokens

AES-256-GCM — encrypting sensitive token data

TOTP — time-based 2FA

Diffie-Hellman — implemented from scratch

System Components

Backend: FastAPI + SQLAlchemy + SQLite

Frontend: Vanilla HTML/CSS/JavaScript

Database: SQLite with user profiles & refresh tokens

API: RESTful endpoints with JWT authentication



```
# keys loading
with open("keys/private.pem", "rb") as f:
    private_pem = f.read()
    PRIVATE_KEY = serialization.load_pem_private_key(private_pem, password=None)

with open("keys/public.pem", "rb") as f:
    public_pem = f.read()
    PUBLIC_KEY = serialization.load_pem_public_key(public_pem)

# Generate and load AES key
def generate_aes_key():
    os.makedirs("keys", exist_ok=True)
    key = secrets.token_bytes(KEY_SIZE)
    with open(AES_KEY_PATH, "wb") as f:
        f.write(key)
    return key
```

```
# Login endpoint
@app.post("/login")
def login(data: LoginSchema):
    with Session(engine) as session:
        user = session.exec(
            select(User).where(User.username == data.username.lower())
        ).first()
```

```
def random_token(nbytes: int = 32) -> str:
    """Generate cryptographically secure random token (hex)"""
    return secrets.token_hex(nbytes)

def random_token_urlsafe(nbytes: int = 32) -> str:
    """Generate cryptographically secure random token (URL-safe)"""
    return secrets.token_urlsafe(nbytes)

def random_bytes(length: int = 32) -> bytes:
    """Generate cryptographically secure random bytes"""
    return secrets.token_bytes(length)

def random_nonce(length: int = 12) -> bytes:
    """Generate random nonce for AES-GCM (default 12 bytes)"""
    return secrets.token_bytes(length)
```

```
# Simple error handler functions
def raise_invalid_credentials():
    raise HTTPException(status_code=401, detail="Invalid username or password")

def raise_invalid_totp():
    raise HTTPException(status_code=400, detail="Invalid TOTP code")
```



Client (Web / CLI)

HTTP

FastAPI Server (Uvicorn)

| Routing & Middleware |

	— JWT Verification
	— TOTP Validation
	— HTTPBearer Auth

API Endpoints

|— GET /, POST /register
|— POST /login, /token/refresh
— POST /logout, GET /users/me

Cryptography & Security

|— RSA-2048 (JWT & Key Wrap)
|— AES-256-GCM (Refresh Tokens)
|— Argon2id (Passwords)
|— TOTP Generation & Verification
— JWT Encode/Decode

Database Layer

|— SQLite (SQLModel)
|— User Table
— RefreshToken Table

Persistent Storage

|— db.sqlite
|— keys/private.pem, public.pem
|— keys/aes_key.bin
— keys/rsa_aes_wrap/



Testing

```
(.venv) PS C:\Users\kaira\secure-auth-system> python test_all_endpoints.py
```

```
=====
TEST: Refresh Token After Logout (Security Test)
=====
```

```
[PASS] Refresh token correctly invalidated after logout: 401
```

```
=====
TEST SUMMARY
=====
```

```
[PASS] Health Check: PASS
[PASS] User Registration: PASS
[PASS] User Login: PASS
[PASS] Get User Profile: PASS
[PASS] Token Refresh: PASS
[PASS] User Logout: PASS
[PASS] Invalid Credentials: PASS
[PASS] Unauthorized Access: PASS
[PASS] Invalid Token: PASS
[PASS] Weak Password Validation: PASS
[PASS] Email Validation: PASS
[PASS] Token Invalidation: PASS
```

```
Total: 12 passed, 0 failed out of 12 tests
```

```
✓ All tests passed!
```

Security Analysis

Strengths:

- ✓ No passwords stored in plain text
- ✓ Short-lived access tokens (15 min)
- ✓ Encrypted refresh tokens in db
- ✓ Input validation & sanitization
- ✓ CORS protection enabled

Additional Recommendations:

- ⚠ Rate limiting on login attempts
- ⚠ HTTPS enforcement (critical for production)
- ⚠ Audit logging for security events

While this system is secure for development, production deployment requires additional hardening like HTTPS, rate limiting, and security headers to prevent abuse and eavesdropping



Challenges Faced

Technical Obstacles:

JWT library confusion (PyJWT vs python-jose) → checked and switched
Token encoding/decoding with RS256 keys → Proper key management
CORS configuration for local development → Debugging
Environment setup & dependencies → Virtual environment isolation
Frontend-backend integration → Comprehensive error handling

Thanks for attention!

